SafeArena: Pitch Deck Presenter Script & Defense Guide

Isolate → Control → Detect → Prove | Hack in Hills '26 | Team: VSV The Tech Giants

Presentation Title:	SafeArena: Controlled Security Testing Environment for AI Systems
Target Pitch Duration:	4 Minutes 30 Seconds (+ Q&A)
Target Audience:	Hackathon Judges, Cybersecurity Architects, AI Safety Engineers
Accompanying Deck:	SafeArena_Premium_Pitch_Deck.pptx (12 Widescreen 16:9 Slides)

Slide 1 — The Hero Slide: SafeArena

Target Timing: **0:00 – 0:30 (30 seconds)**

> "Good morning judges. We are team VSV The Tech Giants, and this is SafeArena. As generative AI and autonomous agents enter production, security teams are tasked with evaluating model guardrails against prompt injection and jailbreaks. But today, adversarial testing without rigorous controls is dangerous: aggressive probes run in flat staging networks can accidentally escape and compromise backend databases or defensive state. SafeArena introduces a purpose-built security testing infrastructure grounded in four core principles: Isolate, Control, Detect, and Prove. We segregate testing into isolated sandboxes, mediate all traffic through a default-deny security gateway, detect prompt-injection threats, and mathematically prove every decision using a SHA-256 audit ledger."

Presenter Guidance & Judge Objective: Hook the judges immediately on the four principles and clarify that SafeArena is an integrated infrastructure, not a generic testing script.

Slide 2 — The Problem: Uncontrolled AI Security Testing

Target Timing: **0:30 – 1:00 (30 seconds)**

> "Why is AI security testing fundamentally different from traditional appsec? First, blast radius. When red teams execute adversarial prompts or multi-step agent jailbreaks, models invoke APIs, tools, and retrieve memory. In an unsegmented environment, a test probe can trigger lateral movement into defended databases. Second, there is rarely an intermediary policy gate: testing tools connect directly to the LLM without access control enforcement. Third, standard application logs are mutable and ephemeral: an operator or compromised system can easily alter or wipe them. Organizations cannot prove to an auditor or regulator what prompts were tested or how the system reacted."

Presenter Guidance & Judge Objective: Establish the three-part problem: uncontained blast radius, missing policy mediation, and lack of non-repudiable audit evidence.

Slide 3 — The Design Gap: Why Existing Approaches Leave a Void

Target Timing: **1:00 – 1:30 (30 seconds)**

> "When teams attempt to solve this today, they face two unsatisfactory extremes. On one side, ad-hoc Python scripts and Jupyter notebooks offer zero network boundaries, no policy checks, and raw text logs. On the other side, enterprise tooling tries to cobble together generic API gateways and cloud SIEMs. But these generic tools cost tens of thousands of dollars, lack AI-specific prompt heuristics, and still store logs in mutable SQL tables vulnerable to root tampering. SafeArena bridges this gap with a purpose-built, self-contained architecture that unites container isolation, default-deny policy mediation, real-time security events, and cryptographic proof into one unified workflow."

Presenter Guidance & Judge Objective: Show fairness to existing approaches while clearly exposing why they leave an evidentiary void in AI testing.

Slide 4 — Our Solution: The SafeArena Framework

Target Timing: **1:30 – 2:00 (30 seconds)**

> "Here is how SafeArena delivers this unified workflow through our four pillars: Number one: ISOLATE. Red attack tooling and Blue defensive assets exist in completely segregated zones, bounded in Docker by internal bridge networks with dropped Linux capabilities and read-only filesystems. Number two: CONTROL. The Security Gateway is the sole bridge between zones, enforcing an uncompromising default-deny stance. Number three: DETECT. Every blocked attempt and heuristic prompt-injection pattern generates a structured, severity-rated security event. Number four: PROVE. Every decision is cryptographically chained using SHA-256 and protected by an atomic head checkpoint."

Presenter Guidance & Judge Objective: Walk through the four pillars rhythmically. Emphasize that each pillar builds directly on the previous one.

Slide 5 — System Architecture: Defense-in-Depth Topology

Target Timing: **2:00 – 2:40 (40 seconds)**

> "Let's look at the architectural blueprint. This is the heart of SafeArena. On the left is the Red Sandbox—unprivileged, running attack tooling on red_net. On the right is the Blue Sandbox—defending internal state on blue_net. At the top is the Client LLM—isolated on llm_net. Notice that red_net and blue_net have NO route to each other and NO internet route. The only bridge is the Security Gateway. Our default policy is strict: Red querying the LLM is ALLOWED through the gateway. Blue querying the LLM is ALLOWED. However, if Red attempts to reach Blue directly, the Gateway terminates the request with HTTP 403 Forbidden. The blocked payload is dropped immediately—it never reaches the destination. Every decision is committed to SQLite for rapid querying and appended to the SHA-256 audit ledger."

Presenter Guidance & Judge Objective: Point to the directional flow badges. Emphasize that the Gateway policy is application-level and Docker internal bridges provide Layer-3 network control.

Slide 6 — How a Request Works: The Request Lifecycle

Target Timing: **2:40 – 3:10 (30 seconds)**

"What happens under the hood during a single test request? Step 1: The test runner posts a payload with declared source and target zones. Step 2: The policy engine evaluates the route against the allow-list. Step 3: If unauthorized, like Red probing Blue, it is immediately rejected with HTTP 403 and the payload is discarded. Step 4: If authorized, such as Red probing the LLM, regex heuristics scan for prompt injection patterns and dispatch the request. Step 5: The gateway canonicalizes the JSON transaction, hashes it with SHA-256, chains it to the previous record, and updates the checkpoint. Step 6: A structured security event is saved to SQLite with a foreign-key pointer to the audit hash, and the dashboard updates in real time."

Presenter Guidance & Judge Objective: Demonstrate the end-to-end speed and deterministic rigor of the evaluation pipeline.

Slide 7 — Security Engineering & Verifiable Trust Architecture

Target Timing: **3:10 – 3:45 (35 seconds)**

"Now let's examine the cryptographic proof. Every transaction is appended to an append-only JSONL file where each record contains the SHA-256 hash of all previous fields. If an attacker or operator modifies even a single character in Record #2—such as changing BLOCKED to ALLOWED—its SHA-256 digest changes completely. When Record #3 is verified, its previous-hash pointer fails, pinpointing the exact line of tampering. Our head checkpoint also detects if records are truncated from the tail. Our built-in tamper test corrupts log copies across 7 distinct scenarios, and our verifier catches 100% of them. And we are honest about threat boundaries: a local hash chain cannot stop an adversary with full OS root access who rewrites both files simultaneously. For production, our roadmap anchors head hashes to external KMS-signed HMACs and WORM cloud storage."

Presenter Guidance & Judge Objective: Highlight the concrete tamper example and showcase technical maturity by openly discussing defensible trust boundaries.

Slide 8 — Product Experience & Live Verification

Target Timing: **3:45 – 4:10 (25 seconds)**

"This is a genuine screenshot of our live running SafeArena console. It is built with zero external CDN dependencies and a strict Content Security Policy. Operators can run one-click scenarios: Red to LLM, Blue to LLM, and blocked cross-zone attempts. When Red attempts to reach Blue, the gateway visual turns red and a high-severity event appears in the real-time filterable feed. Clicking 'Verify audit chain' recalculates thousands of SHA-256 links in milliseconds. And clicking 'Run tamper test' proves our engine by corrupting temporary copies and verifying that all 7 attack vectors are caught. Our entire test suite of 103 automated tests passed in 8.1 seconds."

Presenter Guidance & Judge Objective: Direct the judges' attention to the actual screenshot. Reference real UI metrics (sessions, blocked counts, verified status).

Slide 9 — Technology Stack & Engineering Status

Target Timing: **4:10 – 4:30 (20 seconds)**

"Our implementation is grounded in clean, lightweight, decoupled primitives. The core engines—policy,

audit, events, and LLM interfaces—are completely modular Python 3.12. We built the web API in Flask for zero-dependency offline testability, with a decoupled structure ready for FastAPI. The container topology uses Docker Compose with internal bridge subnets and dropped capabilities. For AI models, we support both a deterministic local mock—with benign, refusal, and canary-leak modes—and upstream Google Gemini API. Crucially, Gemini API keys reside exclusively in gateway memory and are never exposed in API responses or container logs."

Presenter Guidance & Judge Objective: Reassure judges of clean code architecture, portability, and zero secret leakage.

Slide 10 — Target Users & Real-World AI Applications

Target Timing: **4:30 – 4:45 (15 seconds)**

"SafeArena serves four key stakeholders: AI Red Teams who need a blast-proof playground for aggressive jailbreak evaluations. Enterprise AppSec engineers validating multi-agent access boundaries in CI/CD pipelines. AI Governance and compliance officers who need non-repudiable audit logs for the EU AI Act and NIST AI RMF. And universities running hands-on AI security capture-the-flag exercises in a safe, unbreachable lab."

Presenter Guidance & Judge Objective: Rapidly connect technical features to real-world commercial and compliance demand.

Slide 11 — Innovation, Feasibility, and Phased Roadmap

Target Timing: **4:45 – 5:00 (15 seconds)**

"Our engineering roadmap is practical and grounded. Phase 1 is 100% complete: our core prototype with default-deny enforcement, audit chaining, and 103 automated tests. Phase 2 is Docker-ready with internal bridge subnets and automated connectivity probes. In Phase 3, we replace caller-asserted zone headers with mutual TLS and cryptographic JWT attestation. In Phase 4, we scale to enterprise production with KMS hardware signing, WORM storage, and multi-node agent swarms."

Presenter Guidance & Judge Objective: Demonstrate that SafeArena is not just a hackathon prototype, but an architectural stepping stone toward enterprise AI defense.

Slide 12 — Closing: Isolate. Control. Detect. Prove.

Target Timing: **5:00 – 5:15 (15 seconds)**

"In conclusion: SafeArena provides the missing foundation for trustworthy AI security evaluation: Test AI systems in a controlled environment. Enforce the rules. Record what happens. Verify the evidence. We are team VSV The Tech Giants. SafeArena is built, tested, and ready for your evaluation. Thank you judges, and we welcome your questions!"

Presenter Guidance & Judge Objective: Deliver the memorable punchline with confidence, thank the judging panel, and open the floor for Q&A.

Anticipated Judge Questions & Technical Defense

Q1: Is the Red / Blue network separation real, or is it just simulated in software?

SafeArena has two distinct execution modes, and we are completely transparent about which mode is running: In Local Mode (run_windows.bat), it is a logical simulation within a single process to enable rapid offline evaluation. The dashboard explicitly indicates 'Local simulation (policy only)'. In Docker Mode (run_docker.bat), the separation is physical and containerized: red-service, blue-service, and llm-service run on separate internal bridge networks (red_net, blue_net, llm_net) with internal: true. They have no IP routing to each other. Our host-side script scripts/docker_isolation_test.py executes real TCP connection attempts across containers to prove that cross-zone packets are dropped at the Linux bridge level.

Q2: What prevents an attacker in the Red sandbox from spoofing their source identity header?

In our current hackathon prototype, the caller asserts source: 'red' in the JSON body, which is appropriate for a local testbed where the operator controls the test harness. However, in our Docker architecture, network segmentation prevents unauthorized socket routing. As outlined in Phase 3 of our roadmap, production hardening replaces caller-asserted headers with Mutual TLS (mTLS) or cryptographically signed JWT tokens issued during container startup, binding each request to verified cryptographic zone identity.

Q3: Why can't an attacker who compromises the host simply modify the JSONL audit log?

If an attacker modifies any character, deletes a record, or reorders records in the JSONL log, the SHA-256 chain breaks immediately. Because each record embeds the cryptographic hash of the previous record, recomputing hashes detects the exact tampered row. Furthermore, our _head.json checkpoint detects tail-truncation. However, as an honest engineering team, we document that an attacker with full OS root access could rewrite both the log and checkpoint. Our enterprise roadmap solves this via external trust anchors: HMAC keys held in AWS KMS / HSMs and immutable WORM storage (S3 Object Lock).

Q4: How does prompt injection detection work in SafeArena?

SafeArena uses a multi-layered detection approach: 1. Heuristic Inspection: The gateway scans ingress payloads using regex heuristics that detect adversarial prompt patterns, instruction overrides, role-play jailbreaks, and canary exfiltration attempts. 2. LLM Safety Telemetry: When an authorized request reaches the model, the gateway inspects the model's response for refusal states ('I cannot reveal...') or canary token leaks. 3. Severity Classification: Heuristic hits raise MEDIUM severity events, while policy breaches raise HIGH severity, and canary leaks raise CRITICAL.

Q5: Why did you implement the prototype in Flask rather than FastAPI?

Our original MVP was designed for FastAPI, but our build and test environment lacked external internet access to install FastAPI and Pydantic. To ensure our code could be executed and rigorously tested with 100+ unit tests, we built the prototype with Flask and Werkzeug. Crucially, our architecture completely decouples the security logic (policy.py, audit.py, events.py, gateway.py) from the web framework. Porting SafeArena to FastAPI requires only rewriting backend/app.py; every security control, hash chain, and policy check remains identical.